Shadow Shift: A Multi-Agent Grid Maze with Dynamic Shadows and Local Pursuit–Evasion Policies

Sai Dheekshit Reddy Antham, Shreyas Gutti Srinivas, Prasanna Adarsh Kolli

April 14, 2026

Abstract

Objective. To describe an implemented multi-agent grid environment with decentralized policies, partial observability, and a unified evaluation path for interactive and batch experiments.

Approach. The system, called *Shadow Shift*, uses a PEAS (Performance, Environment, Actuators, Sensors) characterization; predators employ graph-based line-of-sight and capped breadth-first path steps; terminal conditions live in one module shared by the graphical simulator and headless trials.

Findings. Representative headless batches under default parameters yield time-based wins in all episodes reported here; automated unit tests pass on fixed mazes. Exported CSV data support plots of outcomes and episode duration.

Keywords: multi-agent systems, pursuit–evasion, grid worlds, PEAS, Python simulation

1 Introduction

Pursuit–evasion games on grids are a standard abstraction for studying search, visibility, and multi-agent coordination [1]. Pursuers and evaders are often modeled with limited sensing and local planners rather than omniscient controllers; simple local rules can still yield rich global behavior. Discrete-time simulators allow identical rules in an interactive renderer and in offline batches, which supports reproducibility.

The deliverable is a maze in which (i) walls, shadow rows, switches, an exit tile, and multiple agents coexist; (ii) predators follow limited line-of-sight and local pathfinding; (iii) neutrals use constrained random walks and may obstruct corridors; (iv) terminal conditions match in the pygame front end and in headless trials. The emphasis is on modular code, repeatable measurements, and tests.

1.1 Multi-agent pursuit–evasion on grids

Xu and Dang [1] study pursuit–evasion on a bounded two-dimensional grid with multi-agent reinforcement learning (MARL). Pursuers achieve very high capture rates after training; emergent cooperation (e.g., flanking) is analyzed via trajectory clustering. We use the same broad setting (grid, multiple pursuers) but hand-designed, inspectable policies instead of learned ones.

1.2 Agent-based tooling

Harwick [2] introduces Helipad, a Python framework for agent-based modeling with spatial support. We do not use Helipad; we use plain Python and pygame [3] for rendering, but follow the same separation of model and view: `world.create_world` feeds both the pygame loop and batch trials [2].

1.3 Scope compared with [1]

Prior work (Xu & Dang).
- Bounded 2D grid; MARL pursuers.
- Composite actions; high capture after training.
- Clustering of trajectories.

Shadow Shift.
- Rule-based predators and neutrals.
- Shadows, switches, exit win.
- Shared `evaluate_round` for GUI and CSV.

2 Methodology

2.1 PEAS characterization

Definition 2.1 (PEAS summary). • **Performance:** survival until capture; win by exit after all switches visited; or win by surviving until horizon T=60 s.

- **Environment:** 20×15 grid, walls, shadow mask, switches, exit.
- **Actuators:** one orthogonal tile per legal step; switches cycle the active shadow row.
- **Sensors:** predators—graph LOS (walls block), shorter range in shadow, memory of last seen cell; neutrals—local occupancy and walkability.

2.2 Software architecture

The model layer (grid, agents, rules) does not import pygame; only `simulation.py` does. Spawns are centralized in `world.create_world`.

Core: `config`, `grid`, `pathfinding`, `agents`, `game_rules`. **Drivers:** `simulation`, `trials`, `plot_results`, `tests/`.

2.3 Discrete-time dynamics

Let $\Delta = 180\,\text{ms}$ be one step. Updates proceed: (1) player; (2) predators; (3) neutrals. The maximum number of steps per episode is

$$N_{\max} = \left\lceil \frac{T}{\Delta} \right\rceil = 334 \quad (T = 60\,\text{s}). \tag{1}$$

2.4 Predators and pathfinding

Predators use `bfs_walkable_depth_map` (LOS, node cap) and `bfs_first_step_toward` with `PREDATOR_BFS_MAX_EXPANSIONS` = 160. The player cell is enterable for capture; other agents block. When the player is invisible, motion mixes jitter, memory bias, and greedy bias.

2.5 Round evaluation

`game_rules.evaluate_round` defines loss (predator on player), exit win (all switches and on exit), and time win (clock $\geq T$). Pygame and `trials.py` call the same function.

2.6 Dependencies

pygame [3]; matplotlib for plots [4]; Python 3.10+.

Table 1: Default hyperparameters (`config.py`).

Quantity	Value
Grid	20×15
T	60 s
Δ	180 ms
Vision (open / shadow)	6 / 3 tiles
BFS cap	160
LOS cap	220 nodes

2.7 Execution workflow

Figure 1 summarizes how the runnable game is wired. Execution begins at `main.py`, which calls `simulation.run` in `simulation.py`. That driver owns the pygame display and the real-time step clock. At startup, `world.create_world` constructs a fresh `Grid` (from `grid.py`) and instantiates the `Player` and `Agent` objects defined in `agents.py`. Those objects hold positions and behavior hooks; the `Grid` holds walls, the shadow mask, switch locations, and the exit tile.

After initialization, the driver repeats a discrete step whenever the accumulated frame time reaches Δ = `STEP_INTERVAL_MS` from `config.py`. Each step applies the same pipeline used throughout this paper: the player moves (keyboard or `auto_move_player`), then `update_agents` advances predators and neutrals in order. Predators consult `pathfinding.py` (BFS depth map and capped shortest-path first step) together with walkability and line-of-sight queries on the shared `Grid`. Finally, `game_rules.evaluate_round` reads the updated positions and grid flags to decide loss (predator on the player), exit win (all switches visited and player on the exit), or time win (clock $\geq T$), or to continue. The dotted arcs indicate that player moves, agent updates, and round evaluation all read the same environment state in `grid.py`; pygame drawing is not shown because it reflects that state and does not alter rules.

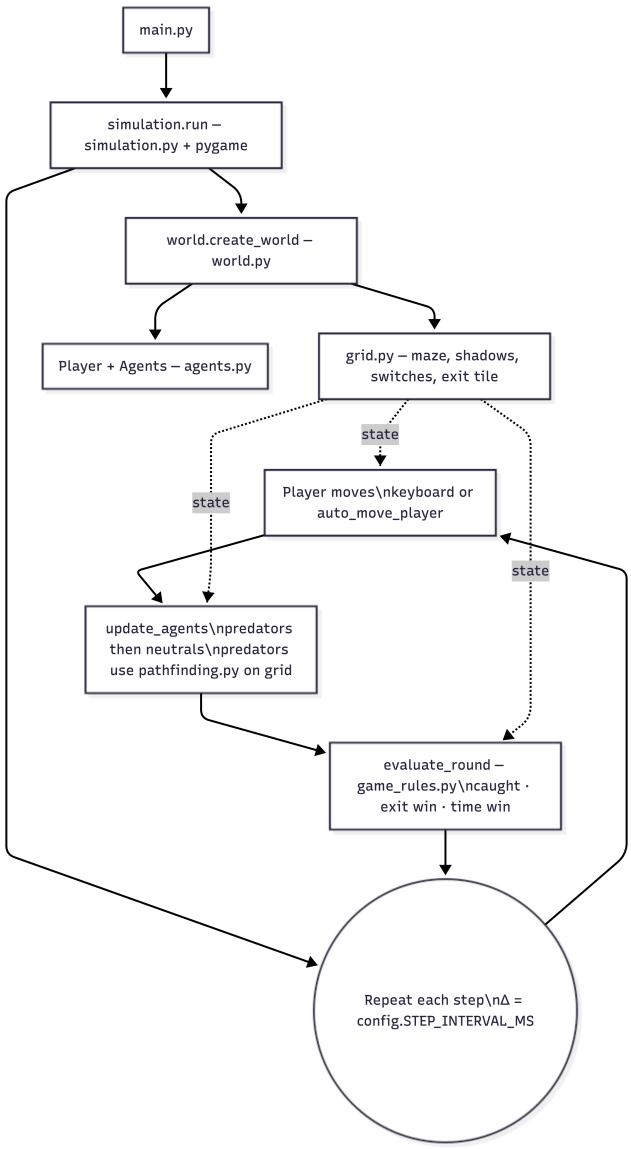

Figure 1: End-to-end execution workflow.

3 Results

3.1 Tests

All seven unit tests pass in the repository state used for this paper.

3.2 Headless aggregates

Table 2 summarizes one exported batch (`results.csv`). Column **Source** names the file; N is the number of episode rows; **Wins** / **Losses** count `outcome` fields (`win` vs. `lose`); **Exit wins** counts wins with `win_kind = exit` when that column exists. This CSV omits `win_kind`, so exit wins are marked `n/a`; winning rows end near `time_sec`$\approx$ 60.12 s, consistent with time-survival wins.

Table 2: Headless trial aggregates from repository `results.csv`.

Source	N	Wins	Losses	Exit wins
`results.csv`	100	90	10	n/a

Reading the table. The ten losses are genuine captures: the autonomous player sometimes shares a tile with a predator before horizon T, so the outcome bar and timeline plots are *not* degenerate for this file. An older draft showed two rows with 500 wins and zero losses; that table reflected a *different* command-line batch where RNG and finite-sample variation happened never to produce a capture. Headless outcomes are stochastic—same code, different seeds or episode counts, can flip the loss column. A non-zero **Exit wins** entry would require logging `win_kind` and completing the switch-and-exit objective.

Remark. Time wins still dominate in `results.csv` (90%), but losses show the predator stack is not harmless; tightening Δ, vision, or maze layout can push the loss rate up further.

3.3 Episode index vs. duration (timeline scatter)

Each point in Figure 2 is one episode row from the trials CSV. The figure encodes three channels simultaneously: batch order (horizontal axis), survival duration (vertical axis), and outcome (color). It is produced as `episode_timeline.png` by `plot_results.py`.

(1) Horizontal axis — `Episode`. The axis is the ordinal position of independent replications (often with `random.seed(seed + i)` per episode in `trials.py`), so left-to-right reads "first episode in the batch, second, ...". It is *not* simulated time inside a single game.

(2) Vertical axis — `time_sec`. The ordinate is the recorded episode-end clock (seconds). High points mean long survival before win, lose, or timeout; low points mean early termination.

(3) Color — outcome category. Matplotlib assigns marker colors from the `outcome` field: wins (green), losses (red), and other outcomes (gray). A single scatter therefore encodes outcome, duration, and batch order jointly.

Reading informal "phases" across the index. Batches sometimes *look* as if the survivor "wins more" early and predators "catch up" in the middle. In this codebase there is *no* gradient-based learning across episodes—policies are fixed rules. Such patterns arise because **(a)** the survivor and predator stacks use *different* algorithms (heuristic scoring and shadow use vs. LOS-limited BFS plus jitter), **(b)** each episode draws fresh stochasticity (neutral motion, tie-breaking), and **(c)** changing the per-episode RNG seed reshuffles those ingredients. Finite runs therefore show streaks that can resemble competition between "learners" even though the mechanism is coupled randomness and rule asymmetry, not online learning. For Table 2, red loss points sit below the green win band near T; an all-win batch would collapse the cloud to a horizontal band at $t \approx T$.

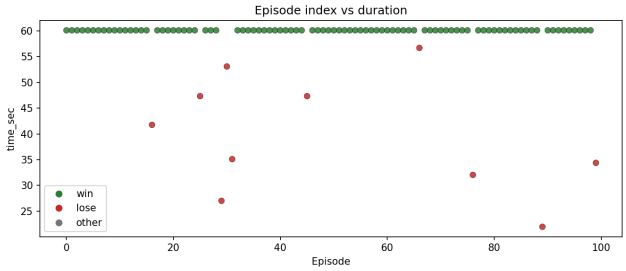

Figure 2: Three-channel batch diagnostic: episode index (x), `time_sec` (y), and outcome color (`episode_timeline.png`).

3.4 Outcome distribution (win vs. lose)

Figure 3 is a *count* summary over a batch of episodes. The horizontal axis lists **outcome labels** produced by `trials.py`—typically `win`, `lose`, and sometimes `timeout` if the step cap is hit before termination. The vertical axis is the **number of episodes** falling in each category.

Under the default autonomous *survivor* policy and maze used for Table 2, wins are overwhelmingly *time wins* (surviving until horizon T) rather than exit wins, so the panel visually collapses to a single dominant bar. The imbalance favors the survivor because the evader policy explicitly scores moves by distance from predators and exploits shadows and switches, while predators operate under *limited* line-of-sight, a capped local BFS, and the same discrete step cadence as the player. The maze geometry spreads pursuit; neutrals add stochastic blocking but do not coordinate capture. Together, these factors make reaching T without co-location on a predator tile easier than rapid corner capture, which is why batch bars skew toward `win` until difficulty parameters (vision, Δ, or maze layout) are tightened.

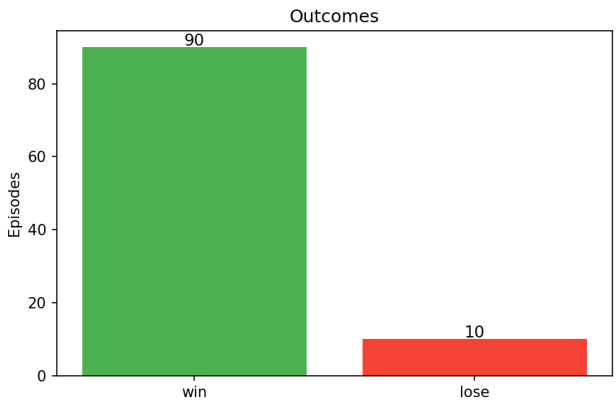

Figure 3: Outcome counts (win / lose / timeout) over the exported batch of episodes (`outcomes.png`).

3.5 Episode duration and steps (linked histograms)

Figure 4 plots the distribution of episode duration as wall-clock `time_sec`, i.e., the simulated clock value

recorded when an episode terminates (win, lose, or time-out). Figure 5 plots the same batch as the distribution of discrete `steps`, the integer step counter incremented by `trials.py` on each iteration. In both histograms the vertical axis is **Count**—the number of episodes falling in each bin.

The two distributions have similar shapes because simulated time advances by a fixed amount per step. With $\Delta = \texttt{STEP_INTERVAL_MS}/1000$ seconds per step, after k steps the clock reads $t \approx k\,\Delta$; thus `time_sec` and `steps` are approximately linked by an affine map,

$$\texttt{time_sec} \approx \texttt{steps}\cdot\Delta \quad (\Delta = 0.18\,\text{s in the default config}). \tag{2}$$

Rescaling the horizontal axis of Figure 5 therefore reproduces Figure 4 up to bin-edge rounding and the exact instant within the last step at which `evaluate_round` fires. When many episodes end on the same time win (near horizon T), both histograms concentrate mass near T and near T/Δ steps respectively.

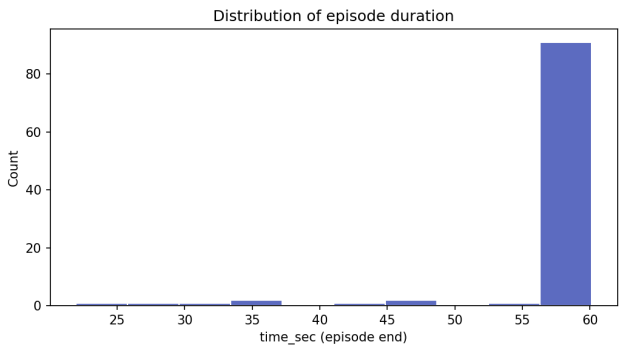

Figure 4: Distribution of episode duration: wall-clock `time_sec` at episode end, with *Count* on the vertical axis (`time_distribution.png`).

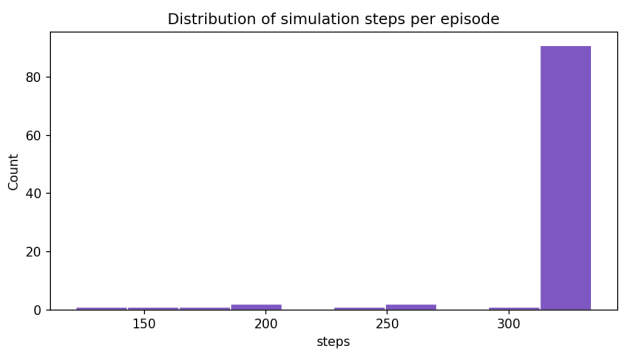

Figure 5: Distribution of simulation steps per episode: discrete `steps` counter, with *Count* on the vertical axis (`steps_distribution.png`).

4 Conclusion

Shadow Shift provides a modular multi-agent maze, one evaluation function for GUI and trials, and automated tests. Extensions include richer maps, learning-based pursuers comparable to [1], and logged trajectories for analysis.

4.1 Limitations

Policies. Predators and the auto-player use fixed heuristics and capped BFS; there is no multi-agent RL or the cooperative pursuit taxonomy analyzed by Xu and Dang [1]. Behavior is therefore not "optimal" in a game-theoretic sense and may not transfer to other mazes without retuning.

Environment class. The model is a 2D orthogonal grid with instant moves, perfect grid snapping, and stylized LOS. It omits velocity, continuous time, sensor noise, and three-dimensional settings noted as out of scope in related work [1].

Stochasticity and reporting. Outcomes depend on seeds and batch length; a loss-free N=500 run and a lossy N=100 export can coexist without code changes. Older CSVs may omit `win_kind`, which weakens reporting of exit vs. time wins.

Evaluation depth. Automated tests cover grid and pathfinding primitives but not full-game integration or performance regression across graphics drivers. No human-subject study establishes playability or perceived difficulty.

Neutrals and asymmetry. Neutral agents do not pursue the player; they only obstruct. Predator count and spawn pattern are fixed in `world.py`, so scalability to larger swarms is unvalidated.

Tooling. The stack is custom Python rather than a dedicated ABM framework such as Helipad [2], so experiment sweeps and visualization hooks require manual scripting.

4.2 Future work

Learning baselines. Train MARL pursuers (or a learned evader) on the same API and compare capture rate and trajectory clusters to [1].

Curricula and difficulty. Automatic schedules over Δ, vision radii, or maze templates to hold a target win rate or mean survival time steady across seeds.

Analysis. Log full trajectories for unsupervised labeling of emergent tactics; export `win_kind` in every CSV; add optional JSON metadata per batch.

Richer worlds. Additional switches, cooldowns, partial observability for the player, or moving shadows to strengthen the exit objective relative to time wins.

Engineering. Deeper `unittest` coverage (predator LOS edges, `evaluate_round` ordering), property-based tests, and CI. Optional integration with Helipad-style hooks for batch sweeps [2].

Human play. Collect manual-play data under `AUTO_PLAYER = False` to compare human vs. heuristic evasion on the same maps.

References

[1] S. Xu and Z. Dang, "Emergent behaviors in multiagent pursuit evasion games within a bounded 2d grid world," *Scientific Reports*, vol. 15, p. 29376, 2025.

[2] C. Harwick, "Helipad: A framework for agent-based modeling in python," *Journal of Open Research Software*, vol. 13, no. 1, p. 25, 2025.

[3] pygame community, "pygame: Python game development library," 2024, available at https://www.pygame.org/.

[4] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.